Corrección de la C02, la instrucción if y digitalRead

Lección 10 — 1MAK, SST-INF-01c — Las dos pelotitas en ocho LED, operadores de comparación, if, leer un pin con digitalRead, sangría

06/11/2026

1 Desde dónde empezamos

La clase tiene dos partes. Primero se **corrige** la tarea para la casa **C02** (las dos pelotitas en ocho LED). Después se da un **paso adelante**: Arduino aprende a **tomar decisiones** con la instrucción `if` y a **leer** un pin con `digitalRead`.

Atención

Lean bien el texto. Antes de escribir el código, lean **todo el texto** del ejercicio, también la parte sobre cómo se entrega. No hagan las cosas a su manera esperando que, por casualidad, coincidan con lo que se pedía.

2 Corrección de la C02: las dos pelotitas

2.1 Qué pedía el texto

“Imagina que cada luz encendida es una **pelotita**. Las dos pelotitas salen de los **dos lados**, van hacia el **centro**, **chocan** y vuelven atrás, repitiendo el movimiento **para siempre**.” Entonces hay **dos luces encendidas a la vez**. En el texto también estaba el dibujo de los primeros cuatro pasos: está dibujada **solo la primera mitad**, después el movimiento vuelve atrás.

2.2 El circuito

El circuito tenía que estar **ya listo** desde la casa: en clase no hace falta volver a hacerlo, hace falta seguir el razonamiento. Se arma así, en pocos minutos:

1. se toma un **Arduino Uno** y una **breadboard** (protoboard);
2. se conectan **enseguida GND y 5 V** a los rieles (*rail*) de la breadboard (negro al menos, rojo al más): así ya está y no se piensa más en eso;
3. se pone un **resistor de 220 Ω** (*resistore*) y un **LED**; después se **seleccionan** los dos componentes juntos, se **copian** y se pegan hasta tener ocho;
4. se agregan enseguida los cables que van del **cátodo** al **GND**: sin ellos el circuito **no puede funcionar**;
5. al final se conectan los pines, empezando desde arriba: **del pin 2 al pin 9**, con un **color distinto** para cada cable.

No hace falta cambiar el color de los LED: hagan como quieran, pero no es ahí donde hay que gastar el tiempo.

Nota

El texto de la tarea usaba los pines **del 3 al 10**; en clase se empezó desde el **2**, hasta el 9. Se puede empezar desde el pin que se quiera: basta **cambiar los números** en el código de forma coherente.

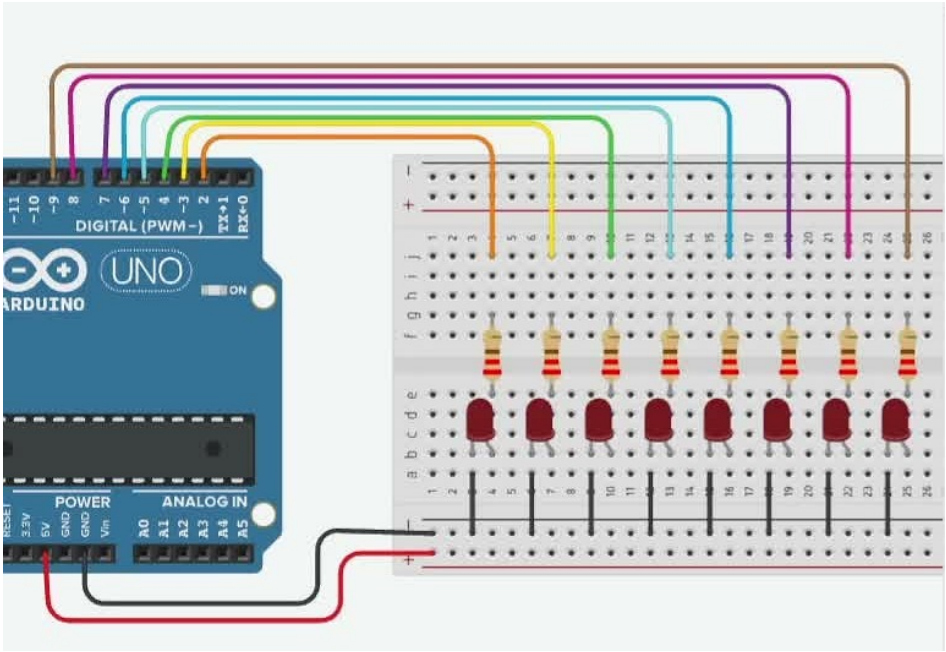


Figura 1: El circuito de la C02 en Tinkercad: ocho cables de colores desde los pines 2-9, ocho resistores de 220 Ω, ocho LED, los cátodos hacia el riel del GND.

2.3 El código, paso a paso

setup. Se **borra** el código de ejemplo que propone Tinkercad. Después se configuran **todos los pines usados** como `OUTPUT` (salida), porque tienen que **manejar LED**: del 2 al 9. La forma más rápida: se escribe `pinMode(2, OUTPUT);`, se hace **copiar y pegar** siete veces y se cambian los números (3, 4, 5, 6, 7, 8, 9).

Paso 1: el primero y el último. Se encienden juntos el pin **2** y el pin **9**, con dos `digitalWrite(..., HIGH)`. Cada paso dura **200 milisegundos**: antes de hacer cualquier otra cosa se espera con `delay(200)`.

Nota

Simulen después de cada cambio. Cada vez que agregan una parte, inicien la simulación y controlen que **hasta ese punto** el código haga lo que tiene que hacer. Los errores se encuentran enseguida, cuando la parte nueva es pequeña.

Paso 2: el segundo y el penúltimo. Primero se **apagan** los dos LED encendidos: los mismos dos `digitalWrite`, pero con `LOW` en lugar de `HIGH`. Justo después, **sin esperar**, se encienden el **3**

y el **8**. Se esperan otros 200 ms y se apagan.

Nota

Ir más lento para controlar. A 200 ms la secuencia es demasiado rápida para controlarla con los ojos. Mientras se escribe el programa conviene poner todos los `delay` en **500 ms**: se ve bien que se encienden primero los LED de afuera y después los de adentro. Al final se vuelven a poner todos en **200 ms**, como pide el texto.

Paso 3: el tercero y el antepenúltimo. Se avanza uno de los dos lados: del 3 se pasa al **4**, del 8 se pasa al **7**. Se encienden el 4 y el 7, se espera, después se apagan **justo el 4 y el 7**.

Atención

Apaguen los mismos LED que encendieron. Si se enciende el 7 pero después, por un descuido, se apaga el 5, en la simulación el LED **7 queda siempre encendido**. Cuando un LED no se apaga nunca, busquen en el código su `digitalWrite(..., LOW)`: casi siempre el número del pin está mal.

Paso 4: las pelotitas en el centro. Se encienden el **5** y el **6**, se espera, se apagan.

Atención

El punto y coma. Cada instrucción (`pinMode`, `digitalWrite`, `delay`) termina con **punto y coma** (*punto e virgola*). Si falta aunque sea uno solo, por ejemplo después de un `delay`, el programa **no arranca**.

La vuelta. Las pelotitas llegaron al centro y ahora tienen que **volver atrás**. No hace falta escribir nada nuevo: basta **repetir el paso 3** (el 4 y el 7) y después **el paso 2** (el 3 y el 8). Se pueden volver a escribir, pero es más rápido **copiarlos y pegarlos** después del paso 4. El paso 1 no se repite al final: de eso se encarga `loop`, que **vuelve a empezar desde el principio** y enciende otra vez el 2 y el 9.

Para que el código se lea mejor se pueden **separar los pasos** con una línea vacía.

```
void setup()
{
  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
  pinMode(7, OUTPUT);
  pinMode(8, OUTPUT);
  pinMode(9, OUTPUT);
}

void loop()
{
  // paso 1: el primero y el último
  digitalWrite(2, HIGH);
  digitalWrite(9, HIGH);
  delay(200);
  digitalWrite(2, LOW);
  digitalWrite(9, LOW);

  // paso 2: el segundo y el penúltimo
  digitalWrite(3, HIGH);
  digitalWrite(8, HIGH);
  delay(200);
  digitalWrite(3, LOW);
  digitalWrite(8, LOW);

  // paso 3: el tercero y el antepenúltimo
  digitalWrite(4, HIGH);
  digitalWrite(7, HIGH);
  delay(200);
  digitalWrite(4, LOW);
  digitalWrite(7, LOW);

  // paso 4: las pelotitas en el centro
  digitalWrite(5, HIGH);
  digitalWrite(6, HIGH);
  delay(200);
  digitalWrite(5, LOW);
  digitalWrite(6, LOW);

  // vuelta: otra vez el paso 3
  digitalWrite(4, HIGH);
  digitalWrite(7, HIGH);
  delay(200);
  digitalWrite(4, LOW);
  digitalWrite(7, LOW);

  // vuelta: otra vez el paso 2
  digitalWrite(3, HIGH);
  digitalWrite(8, HIGH);
  delay(200);
  digitalWrite(3, LOW);
  digitalWrite(8, LOW);
}
```

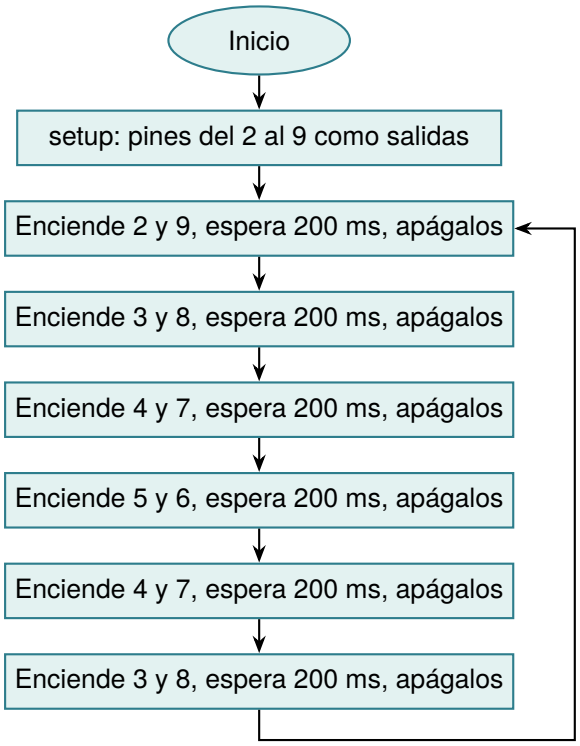


Figura 2: Las dos pelotitas: cuatro pasos hacia el centro, dos para volver, después `loop` vuelve a empezar desde el primer y el último LED.

2.4 Los errores vistos en los bancos

- **El LED "explota" en la simulación.** Si Tinkercad muestra un LED quemado, controlen el **valor de los resistores**: tienen que ser todos de **220 Ω**. Un resistor dejado en **1 Ω** no limita la corriente y el LED se quema. Cuenten también los componentes: 8 LED necesitan **8 resistores**, ni uno más ni uno menos.
- **La secuencia no vuelve atrás.** Si las pelotitas llegan al centro y después vuelven a salir de los lados, falta la **vuelta** (pasos 3 y 2 repetidos). Vuelvan a leer el texto: el movimiento es **de ida y vuelta**.

Nota

El código y el esquema están aquí: intenten volver a hacerlos **en casa** en Tinkercad, para estar seguros de que lo logran solos. No es una entrega: sirve a ustedes, para entender y ver que todo funciona.

Atención

Se trabaja también en casa, y con una computadora. La corrección en clase no es el momento para copiar todo el circuito: el trabajo se hace antes, en casa. En clase no hay tiempo para rehacer todo. El **teléfono** no es una herramienta compatible con Tinkercad: para las tareas hace falta una **computadora**.

3 Las placas que toman decisiones

Hasta ahora conectamos LED a Arduino y los manejamos: con `pinMode` decidimos que un pin es de **salida** (*uscita*), así podemos conectar un LED y decidir **nosotros**, en el programa, si encenderlo o apagarlo. Pero la placa **no interactúa con nosotros**: la encendemos y ella enciende los LED en la secuencia escrita, siempre igual, un poco como las **luces de Navidad**.

Las placas electrónicas también pueden **interactuar** con nosotros y **cambiar su forma de funcionar** según lo que pasa afuera. Piensen en el **microondas**, el **horno**, el **lavarropas**: no funcionan siempre igual. Tienen **botones**, y según lo que apretamos la placa decide si hacer un lavado rápido o un lavado intenso, si calentar o no calentar.

Entonces la placa **toma una decisión**: funcionar de una forma o de otra. Y la toma según una **comparación** entre dos elementos: a partir de ahí decide si hacer una cosa u otra.

4 Los operadores de comparación, otra vez

Los **operadores de comparación** (*operatori di confronto*) ya los vimos en el apunte A04. Atención: el **módulo** `%` **no** es un operador de comparación, porque hace un **cálculo** y da un número. Los operadores de comparación dan **solo verdadero o falso**, y son **seis**:

Operador	Significado
`>`	mayor
`<`	menor
`>=`	mayor o igual
`<=`	menor o igual
`==`	igualdad
`!=`	distinto (desigualdad)

Ejemplo

Creo una variable `int a` con el valor **7** y una variable `int b` con el valor **5**.

- `a > b`: ¿7 es mayor que 5? **Verdadero**.
- `a < b`: ¿7 es menor que 5? **Falso**.

5 La instrucción if

El lenguaje tiene una **estructura** (*costrutto*) hecha justamente para **tomar decisiones** según una comparación. Se llama `if` (en inglés, “si”).

Definición

if — instrucción **condicional**. Se escribe `if`, después entre **paréntesis redondos** la **condición**; abajo, en una línea nueva, se abre una **llave** (*graffa*) y más abajo se cierra. Dentro de las llaves va el código que se tiene que ejecutar.

- Si la condición es **verdadera**, el código dentro de las llaves **se ejecuta**.
- Si la condición es **falsa**, el código dentro de las llaves **se salta**.

```
if (condizione)
{
  digitalWrite(...);
  delay(...);
}
```

El programa, dentro de `loop`, ejecuta sus instrucciones una después de otra: instrucción 1 (por ejemplo un `digitalWrite`), instrucción 2 (un `delay`), instrucción 3, instrucción 4... En un momento hay que **decidir** si ejecutar una parte del código o no, según cómo queremos que se comporte la máquina. Ahí se pone el `if` con su condición: `a > b`, `a == b` o cualquier otra comparación. (En la pizarra la condición estaba escrita en italiano: *condizione*.)

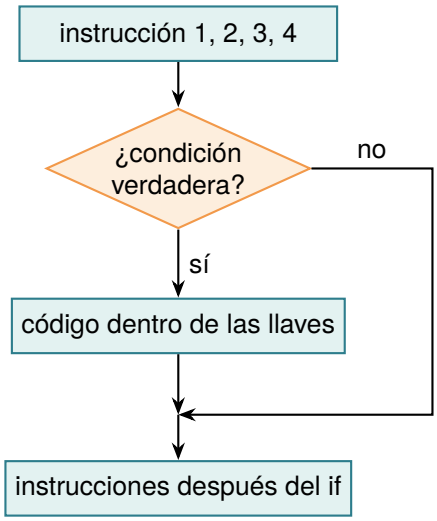


Figura 3: Qué hace el if: si la condición es verdadera ejecuta el bloque entre llaves, si no lo salta.

Atención

Nada de punto y coma después del if. El `if` **no** lleva punto y coma, ni después del paréntesis de la condición ni después de la llave. Vale también para las otras **estructuras** que vamos a ver (por ejemplo los ciclos): en los **comandos** (`pinMode`, `digitalWrite`, `delay`...) el punto y coma **va**, en las **estructuras** **no va**. Muchos se olvidan el punto y coma donde hace falta; aquí es al revés: si lo ponen después de la condición, el if **no funciona** como esperan.

Nota

Por qué el punto y coma arruina el if. `if (a == HIGH);` quiere decir "si `a` está alto, **no hagas nada**": el punto y coma cierra enseguida el if. El bloque entre llaves que viene después ya no depende de la condición y se ejecuta **siempre**. El programa compila igual, así que el error no se señala: se ve solo por el comportamiento equivocado.

6 Los apuntes se estudian: prueba del martes 17/11

Los **operadores de comparación** estaban en el apunte A04: los apuntes (*dispense*) son la memoria de lo que hicimos, y **se tienen que estudiar**, no solo guardar.

Atención

Prueba de firmware (*verifica*): martes 17 de noviembre de 2026 (si el viernes 13 la clase no está lista, se va a mover más adelante). Incluye **todos los apuntes** de firmware, desde el A01 hasta los de la próxima semana: tipos de datos, operadores, Arduino, `setup` y `loop`, `pinMode`, `digitalWrite`, `delay`, operadores de comparación, `if`. La próxima semana se termina el tema y el viernes se hacen ejercicios.

7 Leer un pin: digitalRead

Con un `if` la placa elige si ejecutar o no una parte del código. Pero si las variables las decidimos nosotros al principio del programa (`int a = 7;`), la comparación da **siempre el mismo resultado**, y la elección es siempre la misma. Hace falta una forma de hacer depender la variable de algo que pasa **fuera** de la placa.

Existe un comando que, en lugar de **escribir** en un pin (y encender un LED), lo **lee**. Si para escribir se usa `digitalWrite`, para leer se usa `digitalRead` (con la **R mayúscula**). Entre paréntesis se pone el **número del pin** que se quiere leer.

```
int a;
a = digitalRead(10);
```

Cuando escribimos en un LED, `HIGH` quiere decir encendido y `LOW` apagado. También en lectura un cable puede estar **alto** (`HIGH`, conectado a 5 V) o **bajo** (`LOW`, conectado a 0 V, el GND). `digitalRead` devuelve justamente eso. Así la máquina puede **cambiar su funcionamiento** y elegir si ejecutar o no una parte del código según un **cambio de hardware** hecho en la placa.

Definición

INPUT — un pin que se tiene que **leer** se configura en `setup` como **entrada** (*ingresso*): `pinMode(10, INPUT);`. Los pines de los LED siguen siendo `OUTPUT`.

8 La prueba en Tinkercad: un cable en lugar del botón

8.1 El circuito

1. Un **Arduino** y una breadboard, con **GND y 5 V** conectados a los rieles como siempre.
2. Un **LED** con su resistor de **220 Ω**, conectado al **pin 3**: sirve para **controlar** qué pasa.
3. Un **cable** desde el **pin 10**: es el pin que vamos a leer. La otra punta del cable va a uno de los rieles: a los **5 V** o al **GND**.

8.2 El código

```
void setup()
{
  pinMode(3, OUTPUT);
  pinMode(10, INPUT);
}

void loop()
{
  int a = digitalRead(10);

  if (a == HIGH)
  {
    digitalWrite(3, HIGH);
  }

  if (a == LOW)
  {
    digitalWrite(3, LOW);
  }
}
```

No escribimos mucho: usamos lo que acabamos de ver.

- En `setup`: el **3** es una salida (está el LED), el **10** es una entrada.
- En `loop`: declaramos la variable `a` y le damos el valor leído del **pin 10**.
- Primer `if`: si `a` está **alto**, encendemos el LED. Segundo `if`: si `a` está **bajo**, lo apagamos.

Aquí no usamos una variable `b`: comparamos `a` **directamente** con `HIGH`. Podríamos haber creado una variable `b` con el valor `HIGH`, pero sería un **desperdicio de memoria** sin motivo.

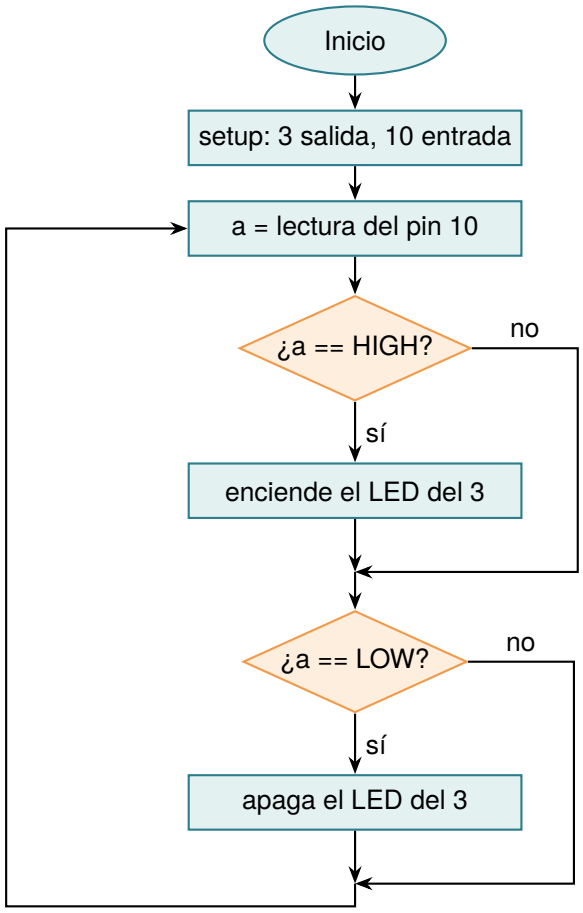


Figura 4: El programa con `digitalRead`: en cada vuelta de `loop` lee el pin 10 y decide si encender o apagar el LED.

8.3 Mover el cable

- El cable del pin 10 va a los **5 V** (el riel rojo): el pin está **alto**, `a` vale `HIGH`, el primer `if` es verdadero y el LED **se enciende**.
- Se **detiene la simulación** (en Tinkercad, mientras la simulación está en marcha, los cables no se pueden mover) y se lleva el cable a los **0 V** (GND): el pin está **bajo**, `a` vale `LOW`, el segundo `if` es verdadero y el LED **queda apagado**.

El **código no cambió**: cambió solo dónde está conectado el cable. Moviendo el cable de 5 V a 0 V decidimos si el LED está encendido o apagado.

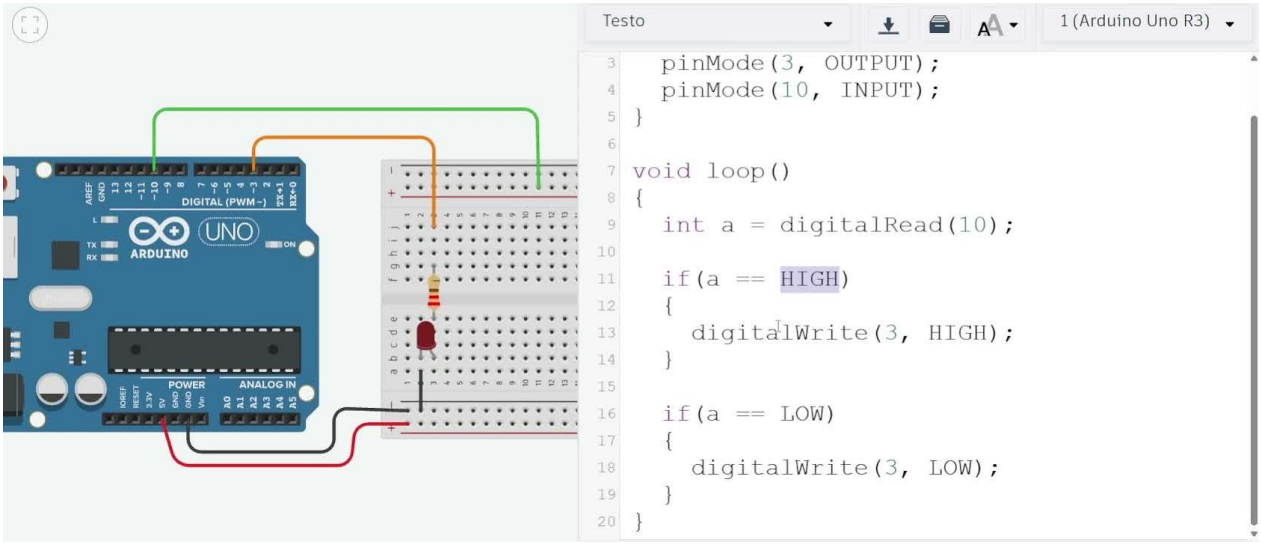


Figura 5: En Tinkercad: el LED en el pin 3, el cable verde desde el pin 10 hacia el riel, y el código con los dos if (primera versión: alto = encendido).

8.4 El funcionamiento lo decidimos nosotros

También se puede hacer al **revés**: encender el LED cuando `a` está **bajo** y apagarlo cuando está **alto**. Basta intercambiar `HIGH` y `LOW` en las dos condiciones.

```
if (a == LOW)
{
  digitalWrite(3, HIGH);
}

if (a == HIGH)
{
  digitalWrite(3, LOW);
}
```

Ahora, con el cable en 0 V el LED **se enciende**, con el cable en 5 V **se apaga**. El funcionamiento **lo decide quien escribe el programa**.

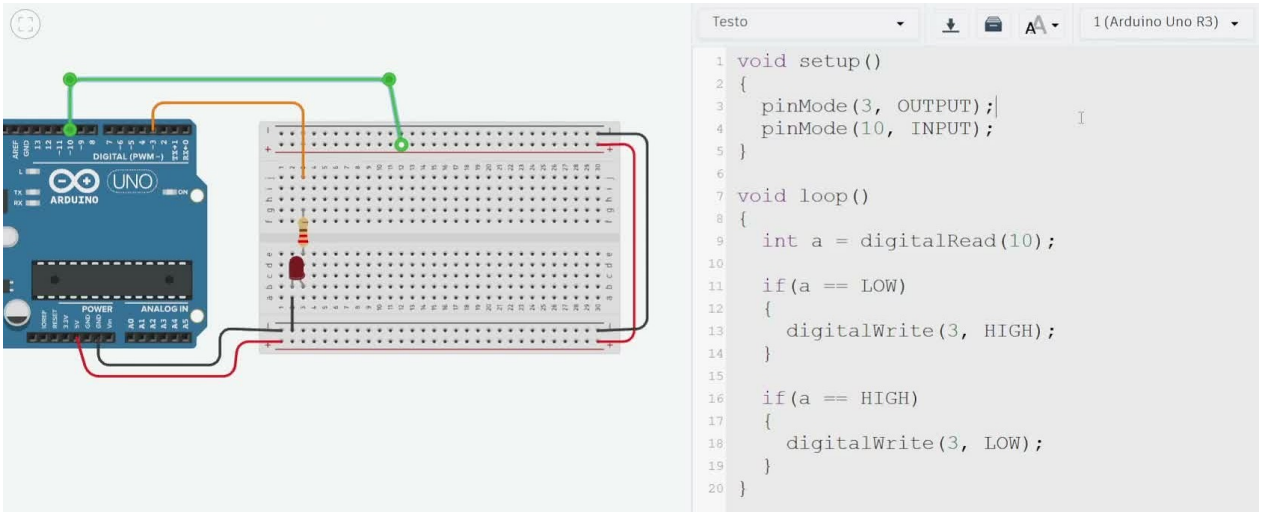


Figura 6: La versión invertida: el LED se enciende cuando el pin 10 está bajo.

Encender un LED moviendo un cable no sirve de mucho: para encender una lamparita no hace falta un Arduino. Pero es la forma más simple de entender **cómo funciona el if**. En lugar del cable, más adelante, habrá un **botón**, como los del microondas o del lavarropas.

Nota

Una entrada no se deja desconectada. El cable del pin 10 tiene que estar siempre conectado a los **5 V** o al **GND**. Un pin de entrada **desconectado** (se dice "flotante") no está ni alto ni bajo: `digitalRead` puede leer `HIGH` o `LOW` **al azar**. Por eso, cuando usemos los botones, también va a hacer falta un resistor que mantenga el pin en un valor preciso cuando el botón no está apretado.

9 El código ordenado: la sangría

Miren el código de arriba: todo lo que está **dentro de un par de llaves** está corrido un poco **a la derecha**. El contenido de `setup` y de `loop` está corrido un paso; el contenido de cada `if`, que está dentro de `loop`, está corrido **un paso más**.

Definición

Sangría (*indentazione*) — cada vez que se abre una llave, es decir, se crea un **contenedor**, todo el texto que está adentro se corre a la derecha. No se usan los espacios: se usa la **tecla de tabulación** (**Tab**), la que está **a la izquierda de la Q**.

Cuántos espacios hace la tecla Tab depende de la **configuración del editor**: en **Tinkercad** la sangría es pequeña, **2 espacios**; en otros editores son 3, 4, a veces 5. Normalmente se puede configurar.

¿Para qué sirve? Para entender **de un vistazo** que una parte del código es un **bloque único**: se ve enseguida dónde empieza y dónde termina cada contenedor. Quien escribe todo **pegado a la izquierda** pierde esta vista: pasa seguido que **pierde una llave** en el camino, que no ve el error o no entiende dónde se está equivocando, porque el código **no está organizado**.

Sin sangría

```
void loop()
{
int a = digitalRead(10);
if (a == HIGH)
{
digitalWrite(3, HIGH);
}
}
```

Con sangría

```
void loop()
{
  int a = digitalRead(10);
  if (a == HIGH)
  {
    digitalWrite(3, HIGH);
  }
}
```

Nota

Algunos editores (por ejemplo el Arduino IDE, con **Ctrl+T**) ordenan la sangría **solos**. El editor de Tinkercad no: ahí se hace **a mano**, con la tecla Tab, mientras se escribe.

10 Qué recordar de esta clase

- **Leer todo el texto** de un ejercicio antes de empezar.
- C02: para pasar de un paso al siguiente se **apagan** los dos LED encendidos y se encienden los dos nuevos; la **vuelta** repite los pasos 3 y 2; `loop` vuelve a empezar desde el paso 1.
- Simular **después de cada cambio**; hacer más lentos los `delay` para controlar, después volver a poner el valor pedido.
- Los operadores de comparación son **seis** y dan **verdadero o falso**; el módulo `%` no es un operador de comparación.
- `if (condición) { ... }`: si la condición es **verdadera** ejecuta el bloque, si es **falsa** lo salta. **Nada de punto y coma** después del if.
- `digitalRead(pin)` **lee** un pin configurado como `INPUT`: `HIGH` si está conectado a 5 V, `LOW` si está conectado al GND.
- El código se escribe **con sangría**, con la tecla **Tab**.
- **Prueba** de firmware: martes **17 de noviembre**, sobre todos los apuntes.

11 Glosario: palabras en italiano que se usan en clase

- **correzione**: corrección.
- **pallina / palline**: pelotita / pelotitas.
- **passo**: paso.
- **filo / fili**: cable / cables.
- **resistore**: resistor.
- **catodo**: cátodo.
- **piedino**: patita, pin.
- **graffa / graffe**: llave / llaves.
- **punto e virgola**: punto y coma.
- **accendere, spegnere**: encender, apagar.
- **avanti, indietro**: adelante, atrás.
- **uscita, ingresso**: salida, entrada.
- **operatori di confronto**: operadores de comparación.
- **maggiore, minore, uguale, diverso**: mayor, menor, igual, distinto.
- **vero, falso**: verdadero, falso.
- **condizione**: condición.
- **costrutto**: estructura (del lenguaje).
- **leggere, scrivere**: leer, escribir.
- **alto, basso**: alto, bajo.
- **indentazione**: sangría.
- **tasto di tabulazione**: tecla de tabulación.
- **dispensa / dispense**: apunte / apuntes.
- **verifica**: prueba, examen.
- **compito**: tarea.